Перша версія розв’язку

Перша основна функція має вигляд:

```
list_pro_main1(L1, L2) ->
    K1 = length(L1) div 2,
    K2 = length(L2) div 2,

    {First1, Second1} = lists:split(K1, L1),
    {First2, Second2} = lists:split(K2, L2),

    Second1 ++ Second2 ++ First1 ++ First2.
```

У цій версії я використовую стандартний підхід. Спочатку знаходжу половину довжини першого списку і половину довжини другого списку. Для цього використовується функція `length`, яка визначає кількість елементів у списку, а оператор `div 2` виконує цілочисельне ділення на 2.

Після цього за допомогою функції `lists:split` кожен список ділиться на дві частини. Наприклад, перший список `L1` розділяється на `First1` і `Second1`, а другий список `L2` розділяється на `First2` і `Second2`.

На останньому рядку новий список формується через оператор ++, який з’єднує списки. Саме тут виконується умова завдання: спочатку додається друга половина першого списку, потім друга половина другого списку, потім перша половина першого і в кінці перша половина другого.

Друга версія розв’язку

Друга основна функція має вигляд:

```
list_pro_main2(L1, L2) ->
    K1 = length(L1) div 2,
    K2 = length(L2) div 2,

    {First1Rev, Second1Rev} = split_halves(L1, 1, K1, [], []),
    {First2Rev, Second2Rev} = split_halves(L2, 1, K2, [], []),

    lists:reverse(Second1Rev) ++
    lists:reverse(Second2Rev) ++
    lists:reverse(First1Rev) ++
    lists:reverse(First2Rev).
```

У цій версії також спочатку визначається межа половини кожного списку. Але далі замість `lists:split` я використовую власну рекурсивну функцію `split_halves`.

Функція `split_halves` проходить по списку поелементно. Вона отримує сам список, номер поточного елемента, межу першої половини та два акумулятори. Один акумулятор призначений для елементів першої половини, а другий для елементів другої половини.

Пояснення рекурсивної функції

Перша частина рекурсивної функції:

```
split_halves([X | Xs], I, Half, AccFirst, AccSecond) when I =< Half ->
    split_halves(Xs, I + 1, Half, [X | AccFirst], AccSecond);
```

означає, що якщо номер поточного елемента ще належить до першої половини списку, тоді цей елемент `X` додається в акумулятор `AccFirst`. Далі функція викликає сама себе для решти списку `Xs`, збільшуючи індекс на 1.

Друга частина:

```
split_halves([X | Xs], I, Half, AccFirst, AccSecond) when I > Half ->
    split_halves(Xs, I + 1, Half, AccFirst, [X | AccSecond]);
```

означає, що коли індекс уже більший за межу половини, елемент додається вже в другий акумулятор `AccSecond`.

Остання частина:

```
split_halves([], _I, _Half, AccFirst, AccSecond) ->
    {AccFirst, AccSecond}.
```

спрацьовує тоді, коли список уже закінчився. У цей момент функція повертає дві накопичені частини списку.